For example, a policy that checks the principal's department using the organization's identity store can dynamically adjust to changes in where an employee is assigned, even if those changes occur day by day. Here's a Cedar policy that encodes that idea:

```
forbid (
    principal,
    action,
    resource in Category::"CriticalResource"
)
unless {
    resource has allowed_department &&
    principal has department &&
    principal.department == resource.allowed_department
};
```

The principal's department is retrieved in real time using the HR system as the data store. The resource's allowed department is stored in a database of resource metadata. If an employee moves from Finance to HR, the policy uses the data to determine if resources the employee is accessing are allowable for an HR employee.

Here are some areas where Policy as Code and Policy as Data complement each other:

- **Scalability**—Policy as code is useful for creating broad, complex rules that apply to many users. Policy as Data can store many fine-grained rules that can be accessed dynamically. These rules are typically created by users themselves as they interact with the software rather than by security administrators or developers.
- **Flexibility**—Policy as Code changes access decisions based on request context. Policy as data makes them based on ever-evolving relationships, roles, and permissions.
- **Security and Compliance**—Policy as Code enforces strict policies based on best practices. Policy as Data allows real-time adjustments without changing policy.
- **Auditability**—Policy as Code offers version control to show how policies have changed. Both can log access decisions to create a